EXP 8	A PYTHON PROGRAM TO IMPLEMENT THE KUHN-TUCKER CONDITIONS LAGRANGIAN MULTIPLIER METHOD

AIM:

The objective of this lab is to introduce and implement the Kuhn-Tucker conditions for constrained optimization using Python. Participants will learn the theoretical basis of the Kuhn-Tucker conditions and apply them to solve optimization problems with inequality constraints.

PROCEDURE:

a. **Objective Function and Constraints:**

Define the objective function and inequality constraints in Python. b. **Kuhn-Tucker Conditions:**

Implement the Kuhn-Tucker conditions using Python functions.

Compute the gradients of the objective function and constraints.

c. **Optimization Setup:**

Use scipy.optimize.minimize or another suitable optimization function.

Incorporate the Kuhn-Tucker conditions as additional constraints or through custom callback functions.

d. **Running the Optimization:**

Execute the optimization procedure and monitor convergence.

Validate results against theoretical expectations.

PROBLEM 1.

Solve the following nonlinear optimization problem and verify the Kuhn-Tucker (KKT) conditions using Python.

Minimize:

$$Z = 2x_1^2 + x_2^2 + x_1x_2 + x_1 + x_2$$

Subject to:

$$x_1^2 + x_2^2 \leq 1$$
$$x_1, x_2 \in \mathbb{R}$$

CODE:

```
from scipy.optimize import minimize
import numpy as np
from scipy.optimize import approx_fprime
# Objective function
def objective(x):
    return 2*x[0]**2 + x[1]**2 + x[0]*x[1] + x[0] + x[1]
# Constraint: x1^2 + x2^2 ≤ 1
def constraint(x):
    return 1 - (x[0]**2 + x[1]**2)
# Initial guess
initial_guess = [0.5, 0.5]
# Define constraint
constraint_obj = {'type': 'ineq', 'fun': constraint}
# Solve optimization
result = minimize(objective, initial_guess, constraints=constraint_obj)

opt_solution = result.x
opt_value = result.fun
```

```
# Compute gradients
gradient = approx_fprime(opt_solution, objective, epsilon=1e-6)
constraint_value = constraint(opt_solution)
constraint_gradient = approx_fprime(opt_solution, constraint,
epsilon=1e-6)
# Print results
print("Optimal Solution:", opt_solution)
print("Optimal Objective Value:", opt_value)
print("Gradient at solution:", gradient)
print("Constraint value:", constraint_value)
print("Constraint gradient:", constraint_gradient)
```

OUTPUT:

Optimal Solution: [-0.14285715 -0.42857143]

Optimal Objective Value: -0.2857142857142856

Gradient at solution: [1.11022302e-10 -2.22044605e-10]

Constraint value: 0.7959183673469388

Constraint gradient: [0.2857143 0.85714286]

PROBLEM 2:

Solve the following nonlinear programming problem using the Lagrangian method and verify the Kuhn–Tucker (KKT) conditions.

Minimize:

$$\boldsymbol{Z = -(2x_1^2 + x_2^2 + x_1x_2 + x_1 + x_2)}$$

Subject to:

$$\boldsymbol{x_1^2 + x_2^2 - 1 \geq 0}$$
$$\boldsymbol{x_1, x_2} \in \mathbb{R}$$

CODE:

```
from scipy.optimize import minimize
```

```
import numpy as np
# Objective function
def objective(x):
    return -(2*x[0]**2 + x[1]**2 + x[0]*x[1] + x[0] + x[1])
# Constraint function (>= 0)
def constraint(x):
    return x[0]**2 + x[1]**2 - 1
# Lagrangian function
def lagrangian(x, lambda_):
    return objective(x) + lambda_ * constraint(x)
# Gradient of Lagrangian
def lagrangian_gradients(x, lambda_):
    grad_objective = np.array([4*x[0] + x[1] + 1,
                               2*x[1] + x[0] + 1])
    grad_constraint = np.array([2*x[0], 2*x[1]])
    return grad_objective + lambda_ * grad_constrain
# KKT Conditions
def kuhn_tucker_lagrangian(solution, lambda_):
    gradients = lagrangian_gradients(solution, lambda_)

    stationarity = np.allclose(gradients, [0, 0], atol=1e-3)
    primal_feasibility = constraint(solution) >= 0
    dual_feasibility = lambda_ >= 0
    complementary_slackness = abs(lambda_ * constraint(solution)) < 1e-
3
    return stationarity, primal_feasibility, dual_feasibility,
complementary_slackness
# Initial guess
```

```
initial_guess = [0.5, 0.5]
# Constraint
constraint_obj = {'type': 'ineq', 'fun': constraint}
# Solve
result = minimize(objective, initial_guess, constraints=constraint_obj)
opt_solution = result.x
opt_value = result.fun
# Incorrect lambda (as per your original logic)
lambda_ = result.fun
# Check KKT
stationarity, primal_feasibility, dual_feasibility, complementary_slackness
= \
    kuhn_tucker_lagrangian(opt_solution, lambda_)
print("Optimal Solution:", opt_solution)
print("Optimal Objective Value:", opt_value)
print("Stationarity:", stationarity)
print("Primal Feasibility:", primal_feasibility)
print("Dual Feasibility:", dual_feasibility)
print("Complementary Slackness:", complementary_slackness)
```

OUTPUT:

Optimal Solution: [5.68355092e+153 3.08302950e+153]

Optimal Objective Value: -9.163312799108927e+307

Stationarity: False

Primal Feasibility: False

Dual Feasibility: False

Complementary Slackness: False

PROBLEM 3:

Solve the following nonlinear programming problem using the Lagrangian Multiplier method.

Maximize:

$$\boldsymbol{Z = x_1 x_2}$$

Subject to:

$$\boldsymbol{x_1^2 + x_2^2 - 10 \geq 0}$$
$$\boldsymbol{x_1 - x_2 = 0}$$
$$\boldsymbol{x_1, x_2} \in \mathbb{R}$$

```
from scipy.optimize import minimize

import numpy as np

# define the objective function and constraints

def objective_function(x):

return -x[0] * x[1]

def constraint1(x):

return x[0]**2 + x[1]**2 - 10

def constraint2(x):

return x[0] - x[1]

# set the initial guess for the optimization

x0 = [1, 1]

# define the Lagrangian function

def lagrangian(x, c1, c2, l1, l2):

return objective_function(x) + l1 * c1(x) + l2 * c2(x)

# set the initial Lagrange multipliers to zero

l1 = 0

l2 = 0

# perform the optimization using the Lagrangian Multiplier Method result
= minimize(lambda x: lagrangian(x, constraint1, constraint2, l1, l2), x0,
```

```
method='SLSQP', constraints=[{'type': 'ineq', 'fun': constraint1}, {'type':
'eq', 'fun': constraint2}])

# print the optimized values and Lagrange multipliers

print('Optimized Values:', result.x)

print('Lagrange Multipliers:', result['jac'])
```

OUTPUT:

Optimized Values: [3.13313166e+147 3.13313166e+147]

Lagrange Multipliers: [-3.13313166e+147 -3.13313166e+147]

RESULT:

Thus the experiment Kuhn Tucker Conditions- Largrangian Multiplier Method has been successfully implemented and executed